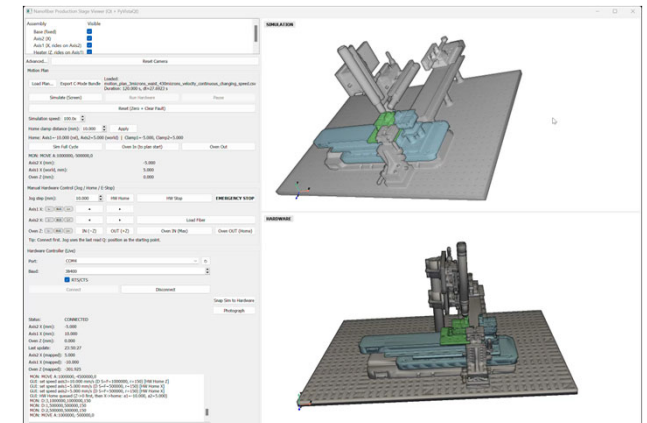
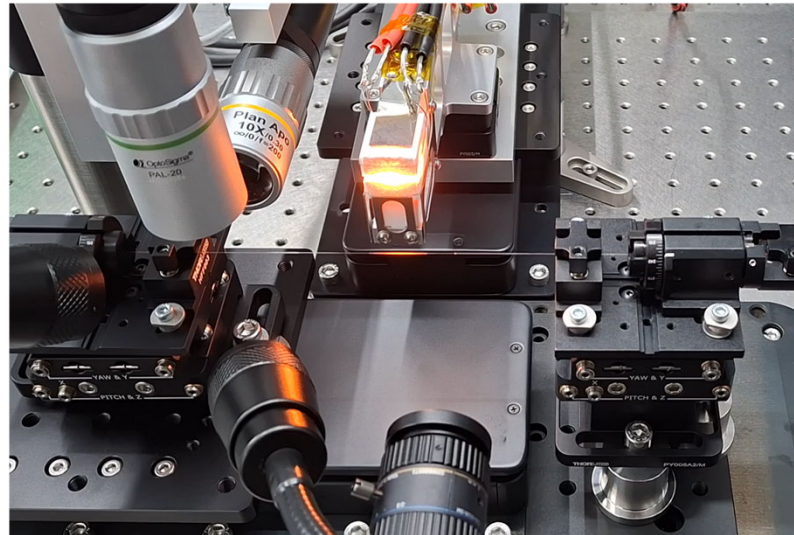
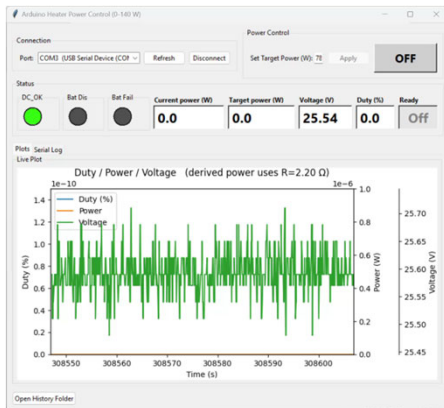




UNIVERSITY OF
WATERLOO



Vibe coded experimental hardware control



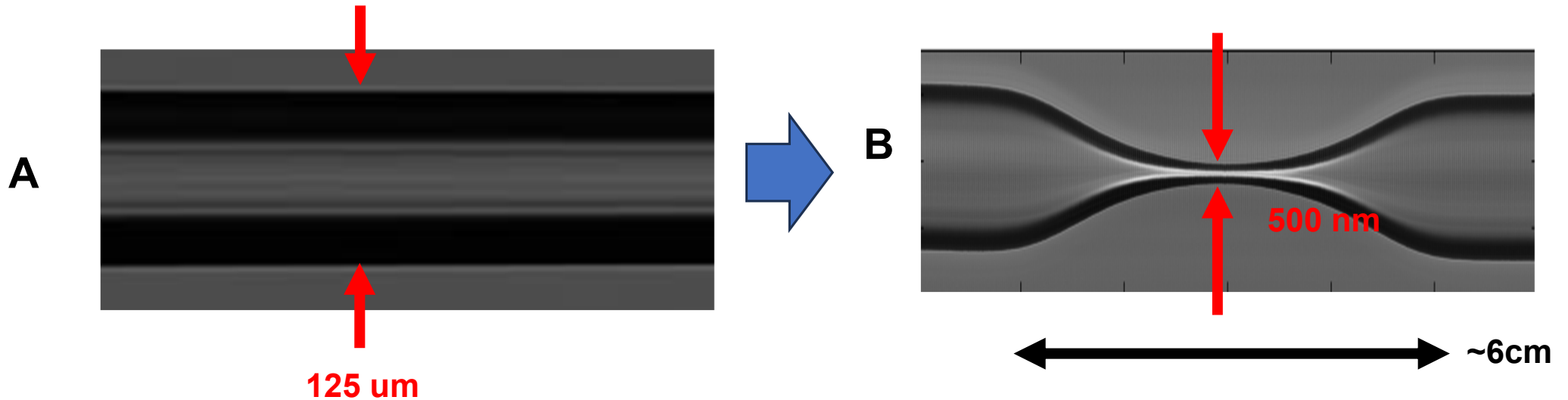
Wen Ting Wang

Wen Ting Wang, Jie-En Chen, Chun Chia Chen, Shayne Bennetts

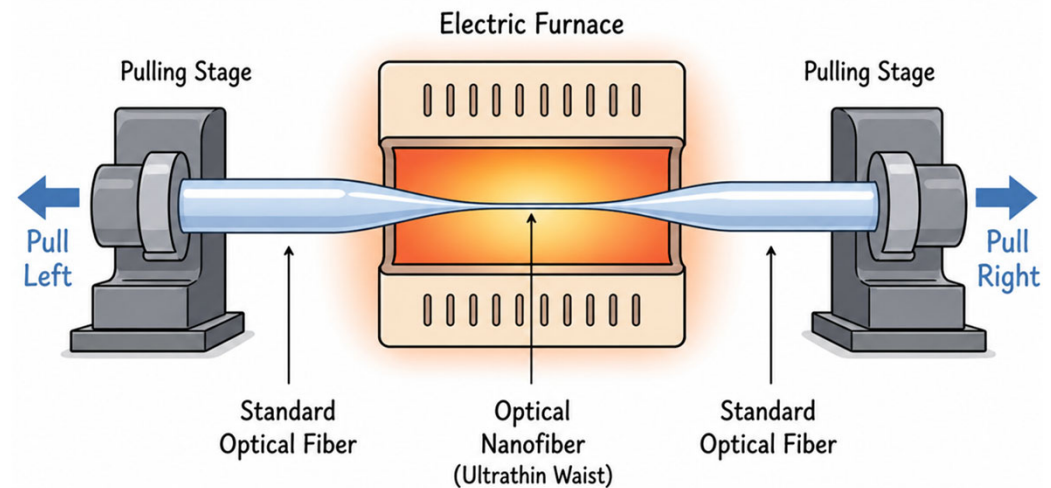
IAMS YbLab 110



Goal: Make optical nanofibers (turn A into B)

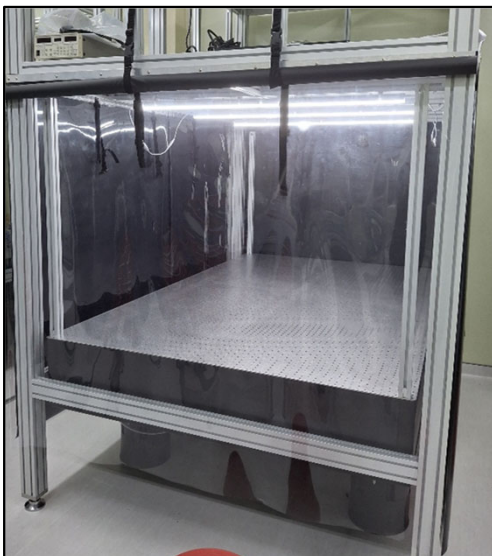


Our approach:
Electric furnace

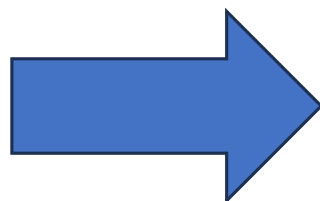




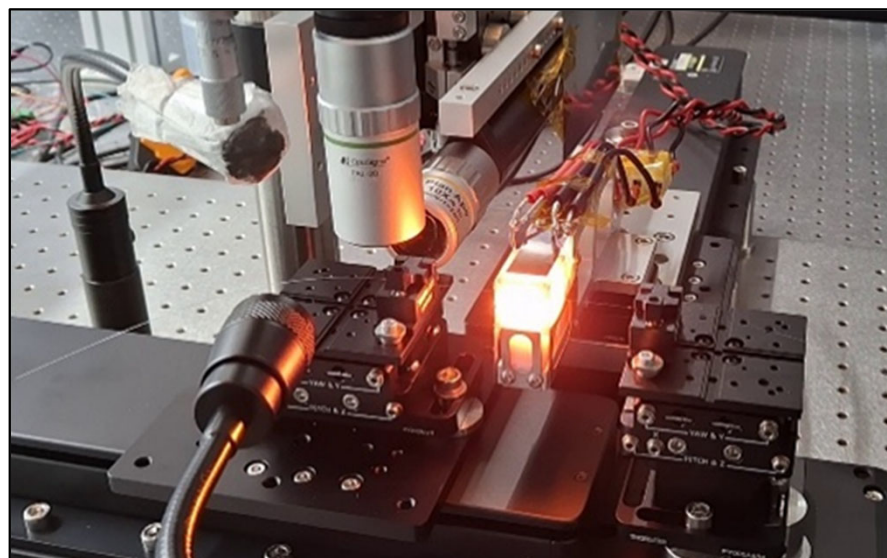
AI made it possible to build this in just 8 weeks



Jan 20 2026



8 weeks



March 20 2026

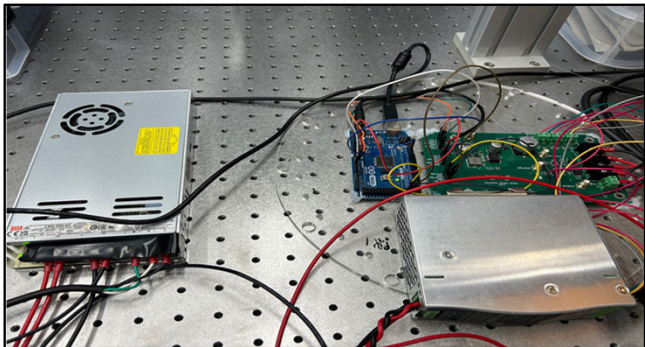




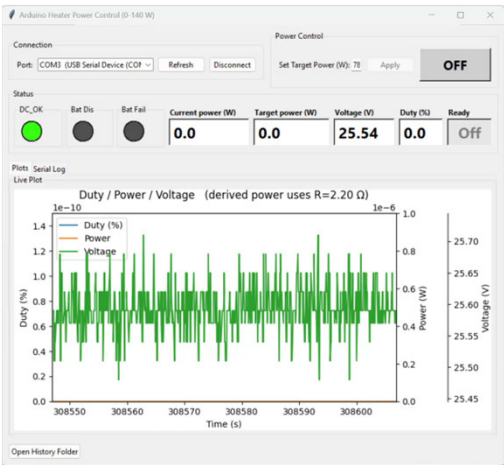
Nanofiber fabrication system



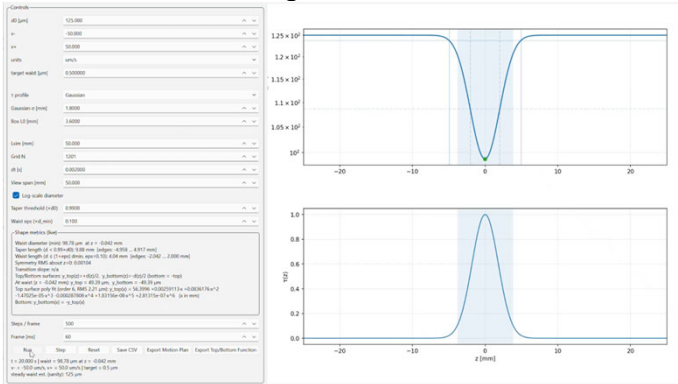
Heater Control System



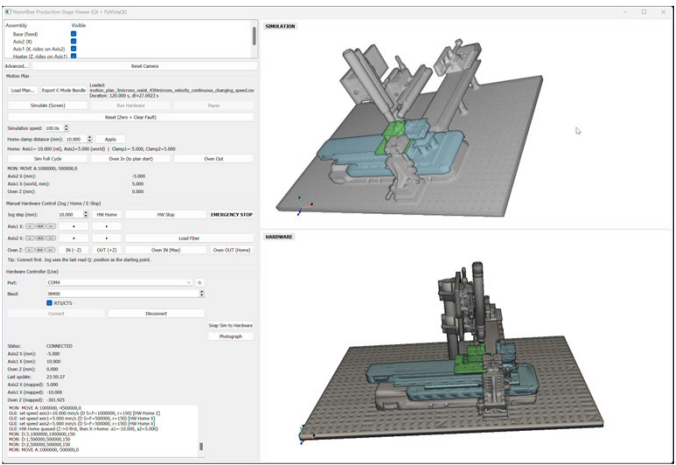
Microcontroller based heater control



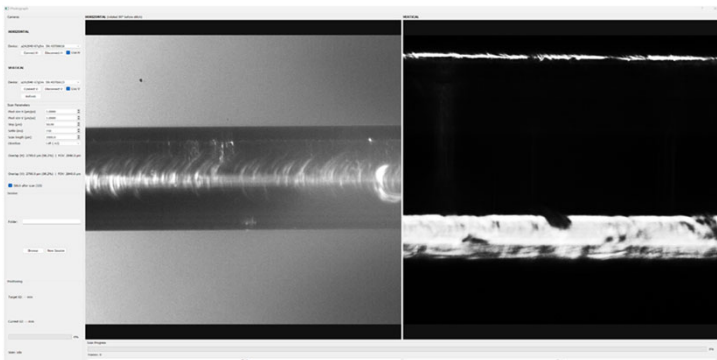
Nanofiber design and simulation



Motion simulation and control



Imaging and characterization





What is “Vibe coding”



1

Describe the problem



Hi, is there a easy way to use python to control the frequency of the laser via moglab wavemeter

Yes! The MOGLabs FZW has a straightforward text-based command interface over TCP/IP or USB, so Python control is quite simple. Here's a minimal example to get you started:

```
python
import socket
import time
```

3

Test it and iterate on bugs

sounds good. i'd like to give it a go. guide me how to download and start integrate it/ useit

Here's a complete guide to get this running. I'll give you all the files you need.

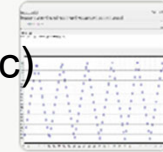
Created 4 files, ran 3 commands, viewed a file >

Download the zip above, then follow these steps:

Step 1 — Install Python (if not already) Download from python.org, version 3.9 or later

2

Detail want you want...(spec)

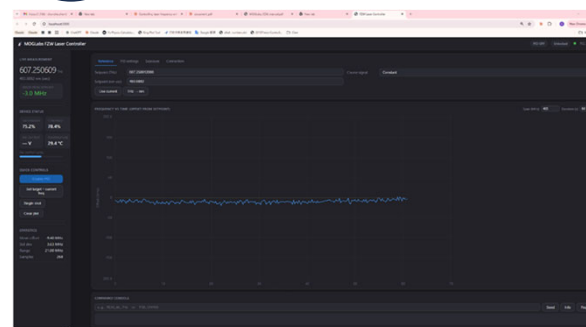


i want to design an interface that I can do the frequency control of the laser, on the GUI interface, i would like to configure the laser PID to control the laser wavelength. a similar feature like highFinesse wavemeter

Let me build you a professional laser frequency control GUI similar to the HighFinesse interface shown.

4

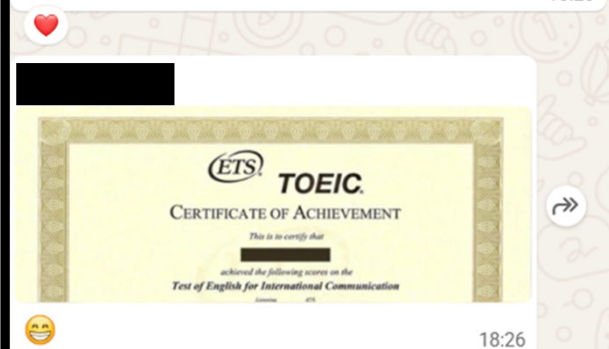
Working GUI interfaced to a wavemeter:



Techniques for coding??? 18:21

I don't have any of it 18:21

I suspect new coding skill in 2026 is called language skill 18:23





“Vibe coding” is a skill

STEP 1

Describe the system, what exists, provide context, boundaries

STEP 2

Write the **specification**: Explain what you want - in detail!

STEP 3

Plan, improve the plan

STEP 4

Code iteration loop: Generate, validate, iterate

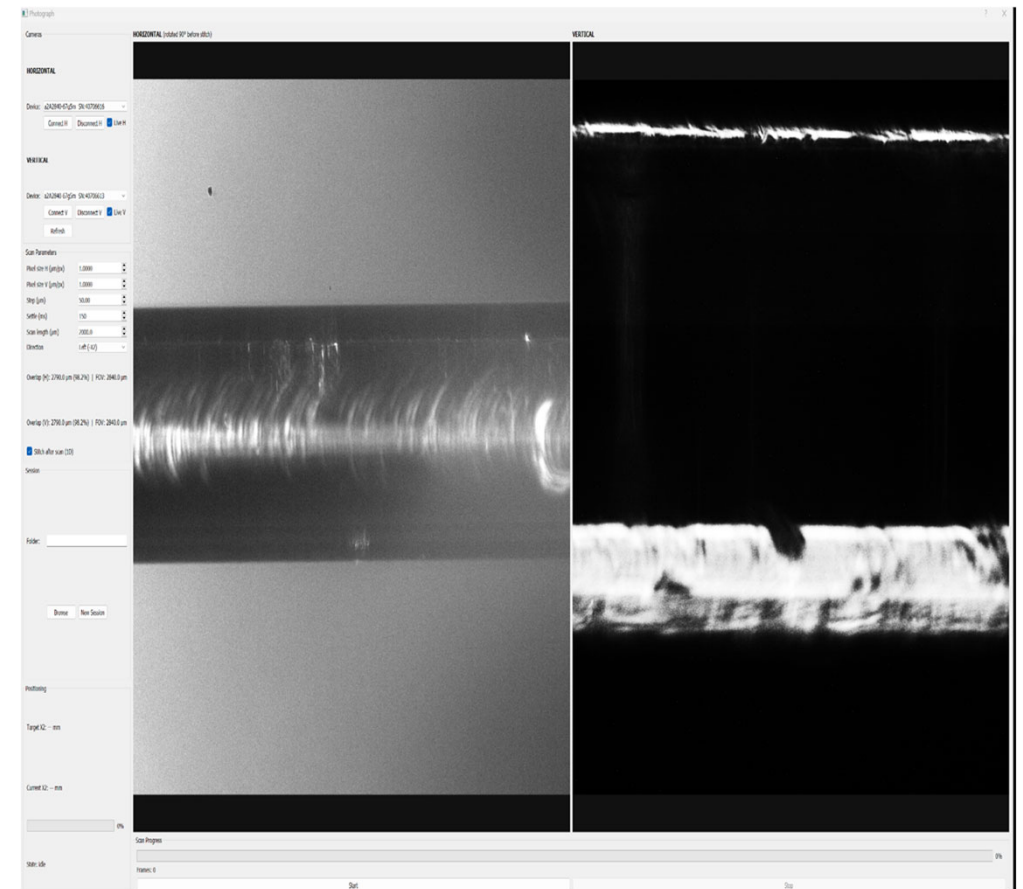
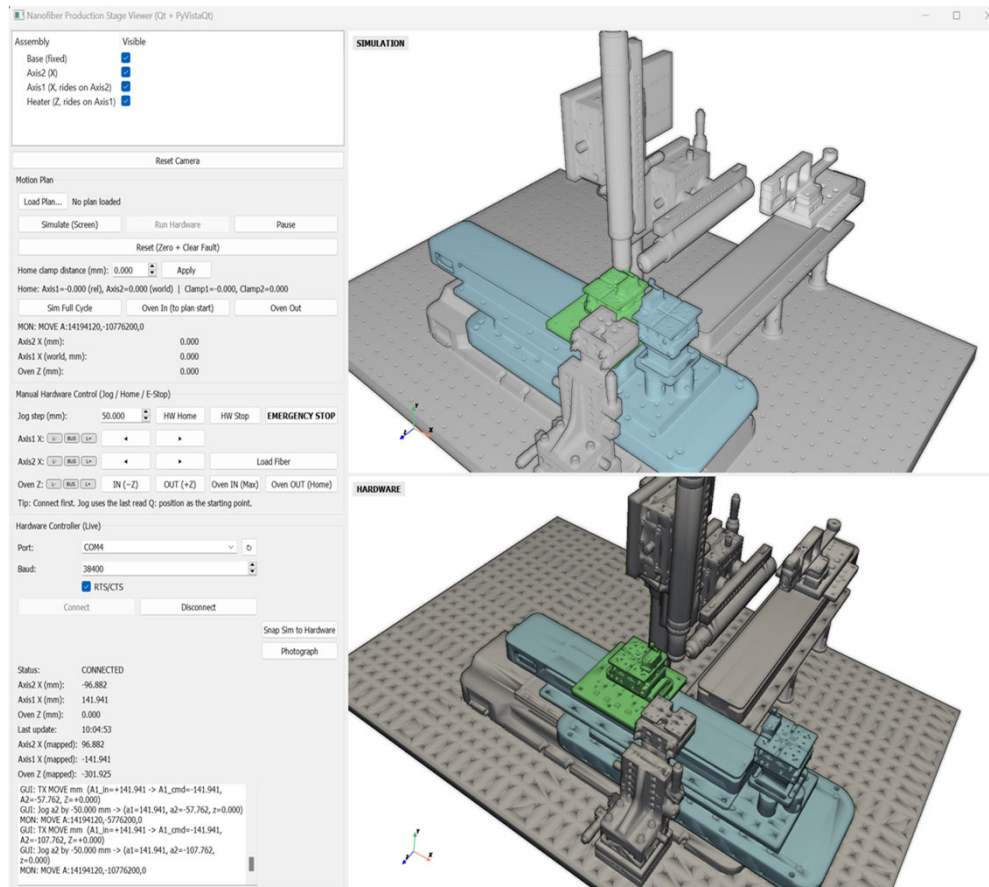
STEP 5

Do not trust – **check** all critical code

Implement features
one at a time



Example System (Stage Viewer)





STEP 1

Describe the system



- Describe the **goal**
- Provide full context (code, papers, interfaces, manuals)
- Describe resources (stages, purpose, subsystems)
- 1st prompts should specify:
 - ❌ No code — provide information only



STEP 1

Describe the system



I am building an automation system for an optical nanofiber fabrication setup using the heat-and-pull method. The goal is to control multiple motorized stages to pull the fiber according to a predefined motion profile and achieve a smooth taper with low transmission loss.

I have uploaded the following to help you understand the system:

- Relevant literature on nanofiber pulling (including taper models)
- My existing simulator code
- Camera interface code for monitoring
- Stage control code (basic movement + communication)

The system involves multiple stages:

- Stage 1 & Stage 2: pulling stages (fiber stretching)
- Stage 3: auxiliary/support stage (alignment or heater positioning)

EXAMPLE



STEP 2

Write the **specification**: Explain what you want



- Specify what you want (as detailed as possible)
- Specify tasks for AI (modules, features, architecture)
-  No code — planning and design only

I want to:

- Load a motion plan from a CSV file (position vs time or velocity profile)
- Use this CSV to automatically control the stages in a synchronized way
- Eventually integrate this into a full automation pipeline (simulation → execution → monitoring)

EXAMPLE



STEP 3

Plan



Before writing any code, I want you to:

1. Fully understand the system and ask questions if anything is unclear
2. Propose a **step-by-step implementation plan** for building this automation system
3. Break down the system into clear modules (e.g., motion control, CSV parser, synchronization, hardware interface, etc.)
4. List all the features that should be implemented (including future extensions like feedback control or vision integration)
5. Suggest a clean and scalable software architecture

Important:

- **Do NOT write any code yet**
- Focus only on planning, structure, and system design
- Be explicit and detailed in how each part should be implemented

EXAMPLE



STEP 3

Check the AI response and **Iterate:**
Improve the plan



1. System Understanding

From your description and provided materials, the system consists of:

- **Motion subsystem:** Multiple motorized stages (Stage 1, Stage 2 for pulling, Stage 3 for support/alignment)
- **Heating subsystem:** External heater with spatial heating profile
- **Sensing subsystem:** Camera providing real-time visual feedback of the fiber
- **Simulation environment:** Existing tool to visualize and validate motion plans before execution
- **Execution input:** CSV-based motion plan defining stage trajectories over time

The goal is to **bridge simulation and hardware execution**, ensuring that a motion plan can be executed reliably and reproducibly on the physical system.

.....

Features List.....

EXAMPLE



STEP 4

Code iteration loop: Generate, validate, iterate



1. Generate the code for the 1st feature/stage of the plan
2. Test the feature
3. Describe any problems and ask the AI to propose a plan to correct them changing nothing else



- State the exact problem (e.g. coordinate mismatch)
- Explain current behavior (simulation vs hardware)
- Ask targeted questions
 - Where to fix?
 - How to design cleanly?
- Focus on system-level solution (no hacks!)



STEP 4

Code iteration loop: Refinement prompts



Coordinate System / Sign Mismatch

I noticed that the motion in simulation and real hardware are not aligned.

In my system:

- The simulator uses a coordinate system where positive X moves to the right
- But on the real hardware, the same command causes motion in the opposite direction

This is causing the execution to be mirrored compared to the simulation.

I want to fix this properly at the system level instead of hacking it locally.

Can you:

- 1.Help me identify where the coordinate transformation should be handled (e.g., stage abstraction layer vs motion plan level)
- 2.Propose a clean way to handle sign inversion or coordinate mapping
- 3.Ensure this solution keeps simulation and hardware consistent going forward

Do not rewrite everything — suggest minimal but scalable changes

EXAMPLE



STEP 5

Do not trust – **check** all critical code

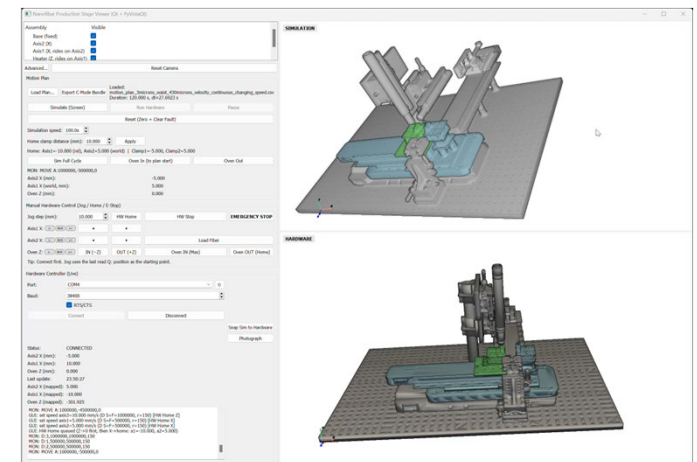


1. We implemented 3D model simulation to check for collisions before running any hardware commands

2. Critical firmware code running the heater was checked by hand

3. Critical firmware code running the heater was checked on a dummy load

4....we still had a bug where code to save last command to EEPROM was written by the AI but was not called in the command loop causing a rare failure mode.





Failure modes to watch out for



1. Alignment mismatch/**insufficient context**: model initially produced code that moved stages step by step to follow an arbitrary path resulting in significant vibration instead of continuous motion
2. **Hallucination**: if there is insufficient context the model can invent, provide manuals/APIs.
3. **Outdated calls/APIs**: training data lags.
4. **Plausible nonsense**: Validate any physics with spot checks and by closing the loop with experimental measurements. Use best models (eg Opus 4.7)
5. **Silent degradation in long sessions**: as context grows, output quality drops. Start fresh chat sessions for new tasks.

Take Home Messages

- Start with a clear problem statement
- Give AI full context upfront
- Force planning before coding
- Break everything into modules
- Refine with targeted prompts
- Then generate, test, and iterate
- Do not trust, check all critical code

